Sri Lanka Tourism Forecasting

Student ID:	KIC-BSCCSAI-251P-034
Programme:	BSc Computer Science with AI
Project Type:	Individual Final Year Project
Focus:	ML Forecasting + Web Deployment
Phases:	13 Phases - Environment to Production

How to Use This Document

Work through each phase in order. Each step tells you WHAT to do and WHY it matters, but you write all the code yourself. When you need explanation on a specific step, bring it to your supervisor or AI assistant with the step number.

Phase Overview

Phase 0	Environment & Project Setup
Phase 1	Data Preparation
Phase 2	Exploratory Data Analysis (EDA)
Phase 3	SARIMA Baseline Model
Phase 4	LSTM Deep Learning Model
Phase 5	Candidate ML Model Comparison
Phase 6	Hybrid Ensemble Model
Phase 7	Evaluation & Statistical Testing

Phase 8	SHAP Explainability
Phase 9	FastAPI Backend
Phase 10	React Web Dashboard
Phase 11	Mobile App (PWA)
Phase 12	Deployment & DevOps
Phase 13	Documentation & Final Submission

PHASE 0 - Environment & Project Setup

Step 0.1 - Verify Python & Packages

Open a terminal (PowerShell or Command Prompt). Run python --version and confirm Python 3.12. Run pip list and confirm that pandas, numpy, scikit-learn, tensorflow, shap, fastapi, and jupyter are all listed. If anything is missing, install it with pip install <package-name>.

Step 0.2 - Fix Your PATH (Important on Windows)

When packages were installed, pip warned that scripts are not on PATH. Go to System Properties ? Environment Variables ? Path and add: C:\Users\KAVIDU ISURA\AppData\Roaming\Python\Python312\Scripts. Close and reopen your terminal. You can now type jupyter notebook directly.

Step 0.3 - Confirm Your Folder Structure

Your project root should have these folders: data/raw/, data/processed/, notebooks/, models/, results/metrics/, results/plots/, results/shap/, backend/data/, frontend/, docs/, scripts/. Create any that are missing manually in File Explorer.

Step 0.4 - Set Up Git Version Control

Open terminal in your project root. Run git init. Create a .gitignore file and add: __pycache__/, *.pyc, .env, node_modules/, .ipynb_checkpoints/. Run git add . then git commit -m 'Initial project setup'. Create a repository on GitHub.com and push your code. Your supervisor can now see your progress as evidence of iterative development.

Step 0.5 - Launch Jupyter

In terminal, navigate to your project root. Run: python -m notebook. This opens Jupyter in your browser. Create all notebooks inside the notebooks/ folder.

PHASE 1 - Data Preparation [notebooks/01_data_prep.ipynb]

Step 1.1 - Load Your Master Dataset

Use pandas to read data/raw/Dataset.xlsx with sheet_name='Dataset'. Print the shape and column names to confirm all 17 columns are present: date, year, month, tourist_arrivals, usd_lkr_rate_monthly, gtrends_flights, gtrends_hotels, gtrends_visa, shock_flag, shock_type, month_sin, month_cos, arrivals_lag_1, arrivals_lag_3, arrivals_lag_12, arrivals_rolling_mean_3, dataset_split.

Step 1.2 - Fix the Date Column

Excel stores dates as integer serial numbers (e.g., 42005 = January 2015). Convert using pd.to_datetime with the base date 1899-12-30 plus timedelta. After conversion, normalise all dates to the 1st of each month using .dt.to_period('M').dt.to_timestamp(). Print min and max dates to verify 2015-2025 range.

Step 1.3 - Fix Data Types

Replace the string 'None' in the shock_type column with actual NaN. Convert all numeric columns to float using pd.to_numeric() with errors='coerce'. Print .isnull().sum() to see how many NaN values exist per column.

Step 1.4 - Verify Feature Engineering

Independently recompute arrivals_lag_1 as tourist_arrivals.shift(1) and compare. Recompute month_sin as sin(2pi × month / 12) using numpy and compare. Recompute arrivals_rolling_mean_3 as .rolling(3).mean() and compare. This confirms your pre-built dataset's calculations are correct before modelling.

Step 1.5 - Validate Shock Flags

Print shock_flag.value_counts() - how many months are flagged as shocks? Print shock_type.value_counts() - verify Easter Attacks, COVID, Economic Crisis are all present. Print dataset_split.value_counts() - verify train/test split proportions.

Step 1.6 - Mark ML-Ready Rows

The first 12 rows have NaN in arrivals_lag_12 (insufficient history). Create a new column ml_ready = 1 where ALL feature columns are non-null, else 0. Print the count of ML-ready rows.

Step 1.7 - Save Processed Dataset

Select columns in a clean order and save to data/processed/tourism_dataset_final.csv. Also save a copy to backend/data/historical.csv - the API will serve this later. Commit to Git: git commit -m 'Phase 1: Data preparation complete'

PHASE 2 - Exploratory Data Analysis (EDA) [notebooks/02_eda.ipynb]

Step 2.1 - Load Processed Data

Load data/processed/tourism_dataset_final.csv using parse_dates=['date'].

Step 2.2 - Time Series Plot with Shock Shading

Plot tourist_arrivals over time using matplotlib. Add red shaded bands wherever shock_flag==1 using ax.axvspan(). Label each shock: Easter Attacks (Apr 2019), COVID-19 (Mar 2020), Economic Crisis (Apr 2022). Save to results/plots/01_arrivals_timeseries.png. This is Figure 1 in your dissertation - visually justifies why a shock-aware model is needed.

Step 2.3 - Seasonal Decomposition

Use statsmodels.tsa.seasonal.seasonal_decompose() with model='additive' and period=12. Plot all four components: Observed, Trend, Seasonal, Residual. Look for the clear 12-month cycle and the downward trend in 2020. Save to results/plots/02_seasonal_decomposition.png.

Step 2.4 - Correlation Heatmap

Select: arrivals, exchange rate, Google Trends (3), shock_flag, lag columns. Compute correlation matrix with .corr(). Plot with seaborn.heatmap(). Save to results/plots/03_correlation_heatmap.png.

Step 2.5 - Exogenous Feature Overlay

Create dual-Y-axis subplots: arrivals vs USD/LKR, arrivals vs gtrends_flights. Observe whether Google Trends leads arrivals by 1-2 months. Save to results/plots/04_exogenous_overlays.png.

Step 2.6 - ACF and PACF Plots

Use statsmodels plot_acf() and plot_pacf() with lags=36. Significant spike at lag 12 confirms annual seasonality - use m=12 in SARIMA. Number of significant PACF spikes suggests AR order. Save to results/plots/05_acf_pacf.png.

Step 2.7 - Monthly Seasonality Boxplot

Filter to stable months only (shock_flag==0). Boxplot with month on X-axis and arrivals on Y-axis. Identify peak season (Dec-Mar) and low season (May-Sep). Save to results/plots/06_monthly_seasonality.png.

Step 2.8 - Write EDA Summary

Add a markdown cell summarising 4-5 key findings. These go directly into your dissertation's EDA section. Commit: git commit -m 'Phase 2: EDA complete, 6 plots saved'

PHASE 3 - SARIMA Baseline Model [notebooks/03_sarima.ipynb]

Step 3.1 - Prepare Train/Test Split

Load processed CSV. Split using dataset_split column: 'train' rows ? train, 'test' rows ? test. Use only tourist_arrivals for SARIMA (it is a univariate model).

Step 3.2 - Fit Auto ARIMA

Import pmdarima and use pm.auto_arima(). Set seasonal=True, m=12. Set stepwise=True and suppress_warnings=True. Fit on training arrivals only. Print the model summary - note the (p,d,q)(P,D,Q,12) order selected.

Step 3.3 - Forecast on Test Period

Use model.predict(n_periods=len(test)) to generate forecasts for the test months.

Step 3.4 - Compute Regime-Segmented Metrics

Calculate RMSE, MAE, MAPE for: full test set, stable months only (shock_flag==0), shock months only (shock_flag==1). Store results in a dictionary for later comparison.

Step 3.5 - Plot SARIMA Forecast vs Actual

Plot actual test arrivals and SARIMA predictions on the same axes with shock shading. Save to results/plots/07_sarima_forecast.png.

Step 3.6 - Save SARIMA Model

Use joblib.dump() to save fitted model to models/sarima_model.pkl. Commit: git commit -m 'Phase 3: SARIMA baseline trained and evaluated'

PHASE 4 - LSTM Model [notebooks/04_lstm.ipynb]

Step 4.1 - Prepare Features and Scaling

Select 11 feature columns + tourist_arrivals as target. Filter to ml_ready==1 rows. Split into train/test using dataset_split. Scale ALL features using MinMaxScaler - fit ONLY on training data, then transform both sets. Scale the target separately with its own scaler. Save both scalers: models/scaler_X.pkl and models/scaler_y.pkl using joblib. The API needs these scalers to scale new inputs before prediction.

Step 4.2 - Build Sequence Windows

Choose lookback window = 12 months (one full year of history). Write a function that creates 3D windows of shape (samples, 12, n_features). Each window is 12 consecutive rows; the label is the arrivals value on the 13th row. Print resulting shapes - you should see (n_samples, 12, 11) for X.

Step 4.3 - Build the LSTM Architecture

Use tensorflow.keras.Sequential with: LSTM(32, activation='tanh'), Dropout(0.2), Dense(16, activation='relu'), Dense(1). Compile with optimizer='adam' and loss='mse'. Print model.summary(). Keep it small - 105 training months means bigger networks overfit.

Step 4.4 - Train the LSTM

Use model.fit() with epochs=200, batch_size=16, validation_split=0.1. Add EarlyStopping(patience=15, restore_best_weights=True). Plot training loss vs validation loss - rising validation loss means overfitting.

Step 4.5 - Generate LSTM Predictions

Predict on BOTH train and test sets using model.predict(). Inverse-transform using scaler_y to get back to real arrival numbers. Save both prediction arrays - train predictions are needed for hybrid stacking in Phase 6.

Step 4.6 - Evaluate LSTM

Compute RMSE, MAE, MAPE - overall, stable-only, shock-only. Add to results dictionary.

Step 4.7 - Save LSTM Model

Use model.save('models/lstm_model.keras'). Commit: git commit -m 'Phase 4: LSTM model trained, scaler saved'

PHASE 5 - Candidate ML Model Comparison [notebooks/05_candidate_ml_comparison

Step 5.1 - Prepare Tabular Feature Set

Load processed CSV, filter ml_ready==1 rows. Feature columns: all 11 exogenous + lag features. Target: tourist_arrivals. Split into train/test using dataset_split column.

Step 5.2 - Define Three Candidate Models

SVR: sklearn.svm.SVR with kernel='rbf', C=10, epsilon=0.1. Random Forest: RandomForestRegressor with n_estimators=300, max_depth=6, random_state=42. Gradient Boosting: GradientBoostingRegressor with n_estimators=300, max_depth=4, learning_rate=0.05, random_state=42. Store all three in a dictionary.

Step 5.3 - Time-Series Cross-Validation

Use TimeSeriesSplit(n_splits=5) - NOT regular KFold. For each candidate: loop through each split, fit on training fold, predict on validation fold, compute RMSE. Average RMSE across 5 folds. Regular KFold shuffles data randomly - for time series you must always train on past and validate on future only. TimeSeriesSplit enforces this.

Step 5.4 - Select the Best Candidate

Compare average validation RMSE for all three models. The lowest average RMSE wins. Print a comparison table: model name | avg val RMSE | rank. Save to results/metrics/candidate_comparison.csv. Save winner name to models/best_model_name.txt. This empirically justifies your model choice in the dissertation.

Step 5.5 - Note the SHAP Implication

If winner is Random Forest or Gradient Boosting ? Phase 8 uses shap.TreeExplainer (fast). If winner is SVR ? Phase 8 uses shap.KernelExplainer (slower, needs a background sample). Write this note in a markdown cell for reference. Commit: git commit -m 'Phase 5: Candidate ML comparison complete, winner selected'

PHASE 6 - Hybrid Ensemble Model [notebooks/06_hybrid_ensemble.ipynb]

Step 6.1 - Load LSTM Predictions

Load the LSTM train and test predictions saved in Phase 4. These are added as an extra feature column called lstm_pred to both datasets.

Step 6.2 - Build the Hybrid Feature Set

Take the ML feature set from Phase 5. Add lstm_pred as an additional column. Your new feature set has 12 columns (11 original + lstm_pred). The stacked ML model can learn when to trust, discount, or correct the LSTM's guess - for example, when shock_flag=1, it may learn to heavily adjust LSTM output.

Step 6.3 - Retrain the Winning Model

Read winner name from models/best_model_name.txt. Instantiate the winning model with original hyperparameters. Fit on full training set (all train rows with all 12 features).

Step 6.4 - Hyperparameter Tuning with GridSearchCV

Define a parameter grid for the winning model. Use GridSearchCV with TimeSeriesSplit(n_splits=5) and scoring='neg_root_mean_squared_error'. Fit on the training set. Print the best parameters found. Your proposal (Section 4.5) requires this for methodological completeness.

Step 6.5 - Generate Final Hybrid Predictions

Use best_estimator_ from GridSearchCV to predict on the test set. These are your final hybrid model predictions.

Step 6.6 - Save the Hybrid Model

Save best estimator to models/hybrid_model.joblib using joblib.dump(). This exact file is what the FastAPI backend will load and serve. Commit: git commit -m 'Phase 6: Hybrid ensemble model saved'

PHASE 7 - Evaluation [notebooks/07_evaluation.ipynb]

Step 7.1 - Collect All Predictions

Load actual test arrivals. Load predictions from SARIMA (Phase 3), LSTM (Phase 4), Hybrid (Phase 6). Load shock_flag for test rows to enable regime segmentation.

Step 7.2 - Compute Comprehensive Metrics

For each model, compute RMSE, MAE, MAPE, R^2 - for all three sub-populations: full test set, stable months only, shock months only. Build a single results table (3 models × 3 regimes × 4 metrics). Save to results/metrics/model_comparison.csv.

Step 7.3 - Diebold-Mariano Statistical Test

Implement the DM test using scipy (required by proposal Section 4.6). It tells you whether the accuracy difference between two models is statistically significant. Run for: SARIMA vs Hybrid (full + shock-only), LSTM vs Hybrid (full + shock-only). Print DM statistic and p-value for each. p-value < 0.05 = statistically significant. How it works: compute error series for each model, compute loss differential d_t = e1² ? e2², then compute t-statistic of d_t using scipy.stats.

Step 7.4 - Forecast Comparison Plot

Plot actual arrivals vs all three model predictions on the same chart with shock shading. Use different colours and line styles for each model. Add a legend. Save to results/plots/08_forecast_comparison.png. This is your key results figure.

Step 7.5 - Error Analysis by Month

Plot prediction errors (actual ? predicted) for hybrid model over the test period. Highlight months where error was largest - were they all shock months? Save to results/plots/09_error_analysis.png. Commit: git commit -m 'Phase 7: Full evaluation with DM test complete'

PHASE 8 - SHAP Explainability [notebooks/08_shap_analysis.ipynb]

Step 8.1 - Load Model and Test Data

Load models/hybrid_model.joblib. Load hybrid test feature set (12 columns with lstm_pred). Read models/best_model_name.txt to know which SHAP explainer to use.

Step 8.2 - Build the SHAP Explainer

If winner is Random Forest or Gradient Boosting: use shap.TreeExplainer(model) - fast and exact. If winner is SVR: use shap.KernelExplainer(model.predict, background_sample) - slow, use ~100 training rows as background data. Compute SHAP values for test set using explainer.shap_values(X_test).

Step 8.3 - Overall SHAP Summary Plot

Use shap.summary_plot() on full test set. This shows each feature's average impact (beeswarm plot). Look for: which features have largest SHAP values - lag_12 (seasonality)? shock_flag? exchange rate? Save to results/shap/shap_summary.png.

Step 8.4 - Shock Period SHAP Plot

Filter test data to shock months only (shock_flag==1). Generate shap.summary_plot() for those rows only. Save to results/shap/shap_shock_periods.png. This directly answers your research question about crisis-period model behaviour.

Step 8.5 - Stable Period SHAP Plot

Same as above but for stable months (shock_flag==0). Save to results/shap/shap_stable_periods.png. Compare with shock plot: during shocks, shock_flag and exchange rate should dominate; during stable periods, seasonal lags should dominate.

Step 8.6 - Export Feature Importance for the API

Compute mean absolute SHAP value per feature. Create a DataFrame with columns: feature, mean_abs_shap. Sort by importance descending. Save to backend/data/shap_importance.csv. The FastAPI /shap-importance endpoint serves this to the web dashboard. Commit: git commit -m 'Phase 8: SHAP analysis complete, importance CSV saved'

PHASE 9 - FastAPI Backend [backend/]

Step 9.1 - Create the Requirements File

Create backend/requirements.txt. List: fastapi, uvicorn[standard], pandas, scikit-learn, joblib, pydantic. This makes it easy to install on any server.

Step 9.2 - Create the Model Loader

Create backend/model_loader.py. Write a get_model() function that loads hybrid_model.joblib once using a global variable. Loading a model on every request is very slow - load once at startup, reuse forever.

Step 9.3 - Create the Data Schemas

Create backend/schemas.py. Define a Pydantic ForecastRequest class with one field for each of the 12 model input features. Define ForecastResponse with forecasted_arrivals float. FastAPI uses Pydantic to automatically validate incoming data and reject malformed requests.

Step 9.4 - Create the Main API File

Create backend/main.py with a FastAPI app instance. Add CORS middleware (allows React frontend on a different port to call the API). Create four endpoints: GET / (health check), POST /predict (returns forecast), GET /history (serves historical.csv as JSON), GET /shap-importance (serves shap_importance.csv).

Step 9.5 - Test the API Locally

Open terminal in backend/ folder. Run: uvicorn main:app --reload --port 8000. Open http://localhost:8000/docs - FastAPI auto-generates interactive documentation. Test each endpoint manually. Screenshot the docs page for your dissertation. Commit: git commit -m 'Phase 9: FastAPI backend complete and tested'

PHASE 10 - React Web Dashboard [frontend/]

Step 10.1 - Create the React App

Open terminal in the frontend/ folder. Run: npx create-react-app . Wait for completion, then run npm start to verify the default page loads at http://localhost:3000.

Step 10.2 - Install Required Libraries

Run: npm install recharts axios. recharts is used for time series and bar charts. axios is for HTTP requests to the backend.

Step 10.3 - Create the API Client

Create frontend/src/api/client.js with three functions: getHistory(), getForecast(), getShapImportance(). Read the API base URL from process.env.REACT_APP_API_URL. Create frontend/.env with: REACT_APP_API_URL=http://localhost:8000.

Step 10.4 - Build ArrivalsChart Component

Create frontend/src/components/ArrivalsChart.jsx. On mount, call getHistory() and store in state. Use recharts LineChart to plot tourist_arrivals over time. Add ReferenceArea elements for shock periods (where shock_flag==1).

Step 10.5 - Build ForecastPanel Component

Create frontend/src/components/ForecastPanel.jsx. Create an input form for each of the 12 model features with sensible defaults. On submit, call getForecast() and display forecasted_arrivals prominently.

Step 10.6 - Build ShapExplainer Component

Create frontend/src/components/ShapExplainer.jsx. Call getShapImportance() on mount. Use recharts BarChart to show mean absolute SHAP per feature as a horizontal bar chart. Sort highest to lowest.

Step 10.7 - Build ShockTimeline Component

Create frontend/src/components/ShockTimeline.jsx. Display a timeline showing when each shock occurred with its label.

Step 10.8 - Assemble the Dashboard Page

Create frontend/src/pages/Dashboard.jsx. Import all four components. Update App.jsx to render the Dashboard. Add page title and professional layout.

Step 10.9 - Styling

Edit frontend/src/index.css for global styles. Use a clean blue/grey colour scheme. Make layout responsive. Add Google Fonts via <link> in public/index.html.

Step 10.10 - Test End-to-End

Start backend: uvicorn main:app --port 8000 (in backend folder). Start frontend: npm start (in frontend folder). Open http://localhost:3000. Verify chart loads, forecast returns prediction, SHAP bars render correctly. Commit: git commit -m 'Phase 10: React dashboard complete'

PHASE 11 - Mobile App (PWA - Stretch Goal)

Step 11.1 - What is a PWA?

A Progressive Web App is a website that can be installed on a phone like a native app. It works offline, sends push notifications, and appears as an icon on the home screen. React apps can become PWAs without writing a separate React Native app.

Step 11.2 - Enable PWA in Your React App

In frontend/src/index.js, change serviceWorker.unregister() to serviceWorker.register(). Edit frontend/public/manifest.json - set app name, theme colour, and icons.

Step 11.3 - Test on Mobile

Run npm run build to create the production bundle. Deploy it (see Phase 12). Open the URL on your phone in Chrome. Tap menu ? 'Add to Home Screen'. Verify it installs like an app and opens without a browser bar.

PHASE 12 - Deployment & DevOps

Step 12.1 - Deployment Architecture

Your app has three parts deployed separately: Frontend (React) ? Vercel (free), Backend (FastAPI) ? Render (free), Models (.joblib) ? stored alongside the backend on Render.

Step 12.2 - Deploy the Backend to Render

Go to render.com and create a free account. Connect your GitHub repository. Create a new Web Service, set root to backend/, set Start Command: uvicorn main:app --host 0.0.0.0 --port 10000. Set Environment to Python 3.12. Click Deploy. Render gives you a public URL like https://your-app.onrender.com. Test at https://your-app.onrender.com/docs. Note: hybrid_model.joblib must be committed to Git for Render to access it. If the file is too large (>100MB), use Git LFS or host on Google Drive.

Step 12.3 - Deploy the Frontend to Vercel

Go to vercel.com and create a free account. Connect your GitHub repository. Set Root Directory to frontend. Vercel detects Create React App automatically. Add environment variable: REACT_APP_API_URL = your Render backend URL. Click Deploy. Vercel gives you a URL like https://your-app.vercel.app.

Step 12.4 - Fix CORS for Production

Update CORS settings in backend/main.py: change allow_origins=['*'] to allow_origins=['https://your-app.vercel.app']. Redeploy the backend.

Step 12.5 - Continuous Deployment (CD)

Both Render and Vercel automatically redeploy whenever you push to GitHub. Your workflow: make changes locally ? git push origin main ? both services redeploy automatically. This is CI/CD - mention it in your dissertation.

Step 12.6 - GitHub Actions (Optional CI Pipeline)

Create .github/workflows/test.yml. Set it up to run notebook tests when you push code. Use pytest for unit tests and nbconvert --execute to test notebooks. This catches errors before they reach production.

Step 12.7 - Basic Monitoring

Render and Vercel provide basic logs and uptime monitoring in their dashboards. Use UptimeRobot (free, uptimerobot.com) to ping your API every 5 minutes and email you if it goes down. Screenshot your deployed dashboard and API docs for the dissertation appendix. Commit: git commit -m 'Phase 12: Deployed to Render + Vercel'

PHASE 13 - Documentation & Final Submission

Step 13.1 - Update README.md

Your project root needs a professional README.md with: project title and description, folder structure overview, how to run locally (step-by-step commands), deployed URLs, technologies used, author and supervisor information.

Step 13.2 - API Documentation

FastAPI auto-generates docs at /docs (Swagger UI) and /redoc. Export or screenshot these for your dissertation appendix. Write a short description of each endpoint in your report.

Step 13.3 - Results Documentation

Ensure results/metrics/model_comparison.csv is complete with all models and all metrics. Create the final results table in your dissertation (copy from CSV into Word/LaTeX). Include all 6 SHAP plots in your report's results chapter.

Step 13.4 - Git History as Evidence

Your commit history on GitHub is evidence of genuine iterative development. Make sure each phase has at least one meaningful commit with a descriptive message. Your supervisor may review your Git log as part of the assessment.

Quick Reference - Technology Stack

Layer	Technology	Purpose
Data processing	pandas, numpy	Load, clean, transform dataset
Visualisation	matplotlib, seaborn	EDA charts and result plots
Baseline model	pmdarima	SARIMA auto-fitting
Deep learning	tensorflow, keras	LSTM sequence model
ML models	scikit-learn	SVR, Random Forest, Gradient Boosting
Explainability	shap	Feature importance analysis
Model persistence	joblib	Save/load trained models
Statistics	scipy	Diebold-Mariano test
API backend	fastapi, uvicorn	Serve predictions over HTTP
Web frontend	React, recharts, axios	Interactive dashboard
Backend deploy	Render	Free cloud hosting
Frontend deploy	Vercel	Free React hosting
Version control	Git, GitHub	Code history and backup
CI/CD	Render + Vercel auto-deploy	Automatic deployment on push

Git Commit Checklist

- [ ] git commit -m "Initial project setup"
- [ ] git commit -m "Phase 1: Data preparation complete"
- [ ] git commit -m "Phase 2: EDA complete, plots saved"
- [ ] git commit -m "Phase 3: SARIMA baseline trained"
- [ ] git commit -m "Phase 4: LSTM model trained"
- [ ] git commit -m "Phase 5: Candidate ML comparison complete"
- [ ] git commit -m "Phase 6: Hybrid ensemble model saved"
- [ ] git commit -m "Phase 7: Evaluation with DM test complete"
- [ ] git commit -m "Phase 8: SHAP analysis complete"
- [ ] git commit -m "Phase 9: FastAPI backend complete"
- [ ] git commit -m "Phase 10: React dashboard complete"
- [ ] git commit -m "Phase 11: PWA mobile support (if time permits)"
- [ ] git commit -m "Phase 12: Deployed to Render + Vercel"
- [ ] git commit -m "Phase 13: README and final documentation"